========== Multivariate Regime-Conditional Simulator ==========

Simulated World Project.

Part 1.2: Synthetic Worlds. Regime SImulation, Factor Score

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
from sklearn.preprocessing import StandardScaler
from hmmlearn.hmm import GaussianHMM
from scipy import stats
from sklearn.neighbors import KernelDensity
```

In [9]:
```
# Load data
data = pd.read_csv("project_DATA/features_for_part_1_2_df.csv", index_col="Date", parse_dates=True)
df = data.copy(deep=True)
```

In [10]:
```
################ Check if any NaN exists in the entire DataFrame #########################
def check_nan(df):
    nan_mask = df.isna()
    has_nan = nan_mask.any().any()
    print(f"Does the DataFrame contain any NaN values? {has_nan}")
    if has_nan:
        nan_counts = nan_mask.sum()
        print(f"Count of NaN values per column:\n{nan_counts[nan_counts > 0]}")
##########################################################################################
```

In [11]:
```
check_nan(df)
# Clean
df = df.dropna()
```

```
Does the DataFrame contain any NaN values? False
```

In [12]:
```
# Prepare Target & Regimes
# Forward returns (prediction target)
df.rename(columns={'target':'return'}, inplace=True)
df['fwd_return'] = df['return'].shift(-1)
```

In [13]:
```
# Create Market State Variables for GaussianHMM

# 1. Shock
df['log_ret'] = np.log1p(df['return'])

# 2. Volatility
# shift(1) means: Regime is then fully known at decision time. Zero leakage concerns
df['realized_vol'] = (df['return'].ewm(span=20).std().shift(1))

# 3. VIX
df['VIX_level'] = df['VIX_indx_close']
df['VIX_change'] = np.log(df['VIX_indx_close']).diff()

# 4. Volume
df['vol_z'] = (
    (df['Volume'] - df['Volume'].rolling(20).mean()) /
    df['Volume'].rolling(20).std()
).shift(1)

df = df.dropna()
```

In [14]:
```
hmm_features_df = df.copy(deep=True)
```

In [15]:
```
hmm_features_df.shape
```

Out[15]:
```
(6468, 66)
```

In [16]:
```
# List of columns to define regime
state_cols = ['log_ret','realized_vol', 'VIX_level', 'VIX_change', 'vol_z']
```

In [17]:
```
# Scale
scaler = StandardScaler()
X_state = scaler.fit_transform(hmm_features_df[state_cols].values)
```

In [18]:
```
# Run the model
model = GaussianHMM(
    n_components=3,            # 3 regimes
    covariance_type="full",
    n_iter=1000,
    random_state=42
)

model.fit(X_state)
```

Out[18]:

▾ GaussianHMM

▸ Parameters

n_components	3
covariance_type	'full'
min_covar	0.001
startprob_prior	1.0
transmat_prior	1.0
means_prior	0
means_weight	0
covars_prior	0.01
covars_weight	1
algorithm	'viterbi'
random_state	42
n_iter	1000
tol	0.01
verbose	False
params	'stmc'
init_params	'stmc'
implementation	'log'

In [19]:

```
# Predict the regimes
hidden_states = model.predict(X_state)
# Create Regime Factor
hmm_features_df['hmm_regime'] = hidden_states
```

In [20]:

```
# Regime stats
hmm_summary = hmm_features_df.groupby('hmm_regime')[state_cols].mean()
print(hmm_summary)
```

```
              log_ret  realized_vol  VIX_level  VIX_change     vol_z
hmm_regime
0            0.000716      0.006079  13.298359   -0.000108 -0.021610
1            0.000107      0.009715  19.756116    0.000408  0.049027
2           -0.000405      0.019904  32.397571   -0.001195  0.065138
```

In [22]:
```
# Check persistence
print(model.transmat_)
# Save the matrix. We'll use it to simulate regimes

HMM_matrix = model.transmat_
# Save to file
np.save('project_DATA/HMM_real_world_transition_matrix', HMM_matrix)
```

```
[[9.79538250e-01 2.04617504e-02 1.06263579e-90]
 [1.80061367e-02 9.72568467e-01 9.42539597e-03]
 [1.87665236e-58 2.16112139e-02 9.78388786e-01]]
```

In [23]:
```
# HMM regimes are unlabeled. Lets label them
mapping = {0:'calm',1:'transition',2:'stress'}
hmm_features_df['regime_name'] = hmm_features_df['hmm_regime'].map(mapping)
```

In [24]:
```
hmm_features_df.shape
```

Out[24]:
```
(6468, 68)
```

In [ ]:
```
#==========================================================================================================
```

Visualize the Close prices for each regime

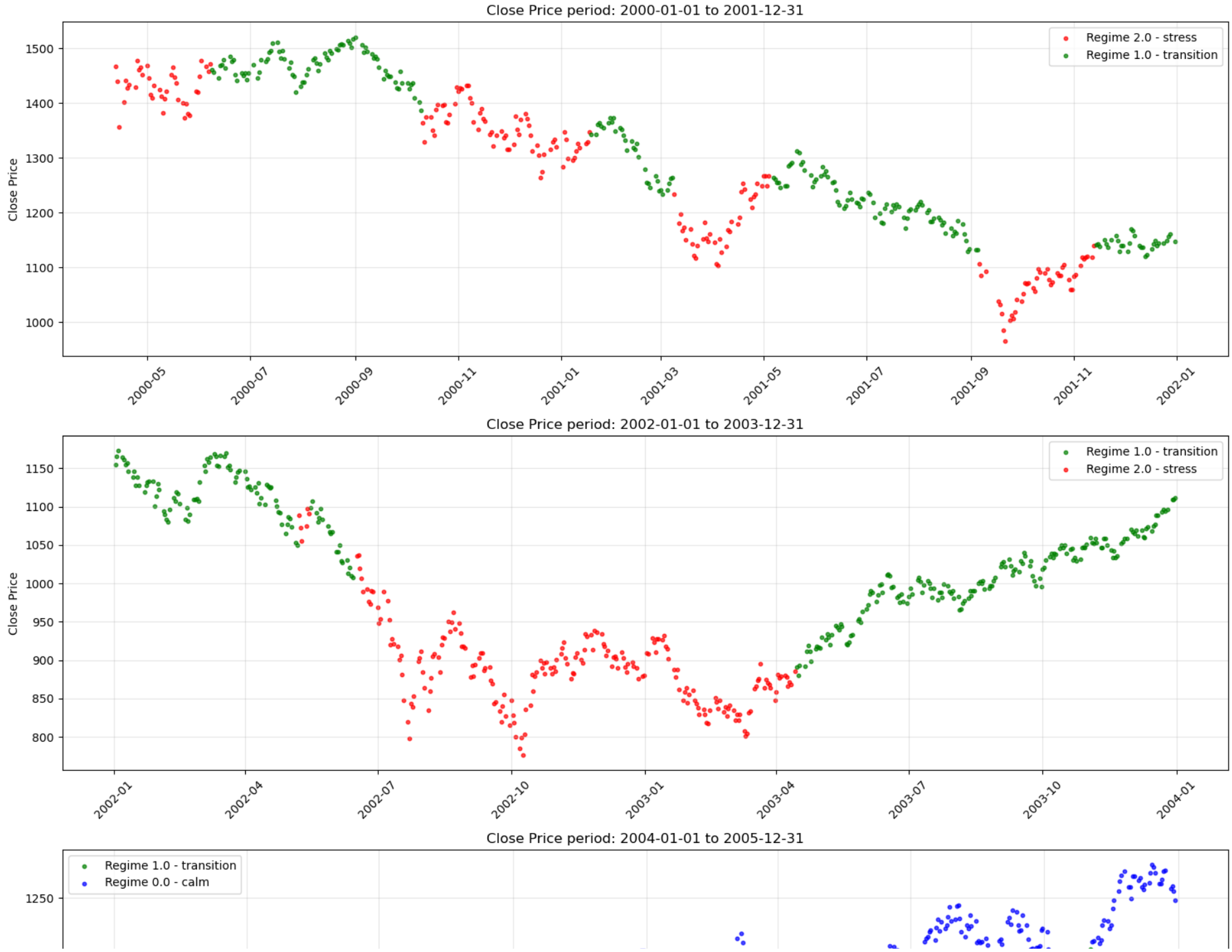
In [25]:
```
# Load data
close_data = pd.read_csv("project_DATA/CLOSE_data_for_part_1_2.csv", index_col="Date", parse_dates=True)
close_df = close_data.copy(deep=True)
```

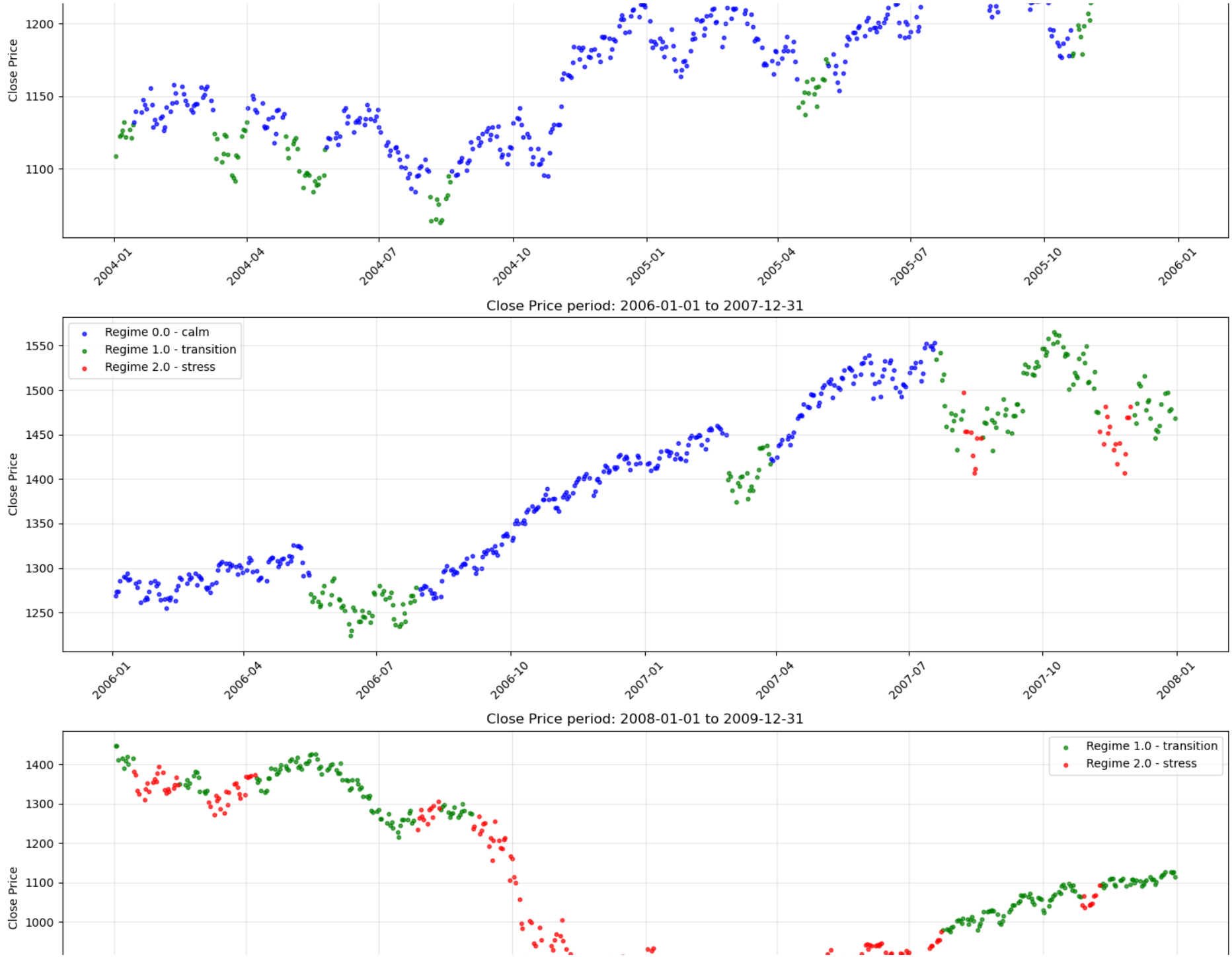
In [26]:
```
hmm_regime_df = hmm_features_df[['hmm_regime']]
```

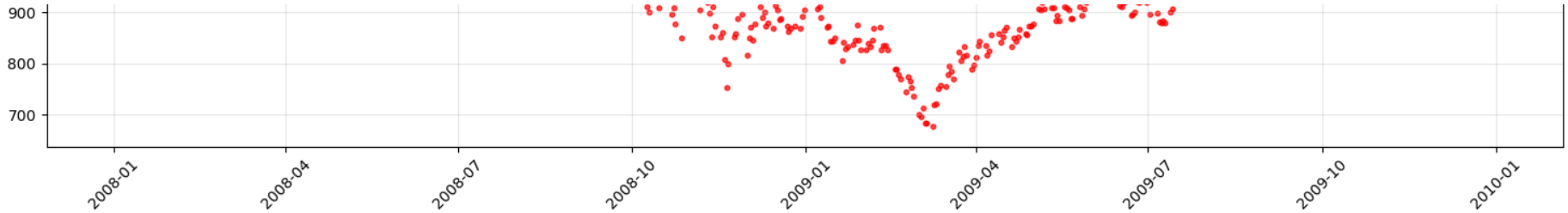
In [27]:
```
# Align the data (make sure dates match)
combined_df = pd.concat([close_df, hmm_regime_df], axis=1).dropna()
```

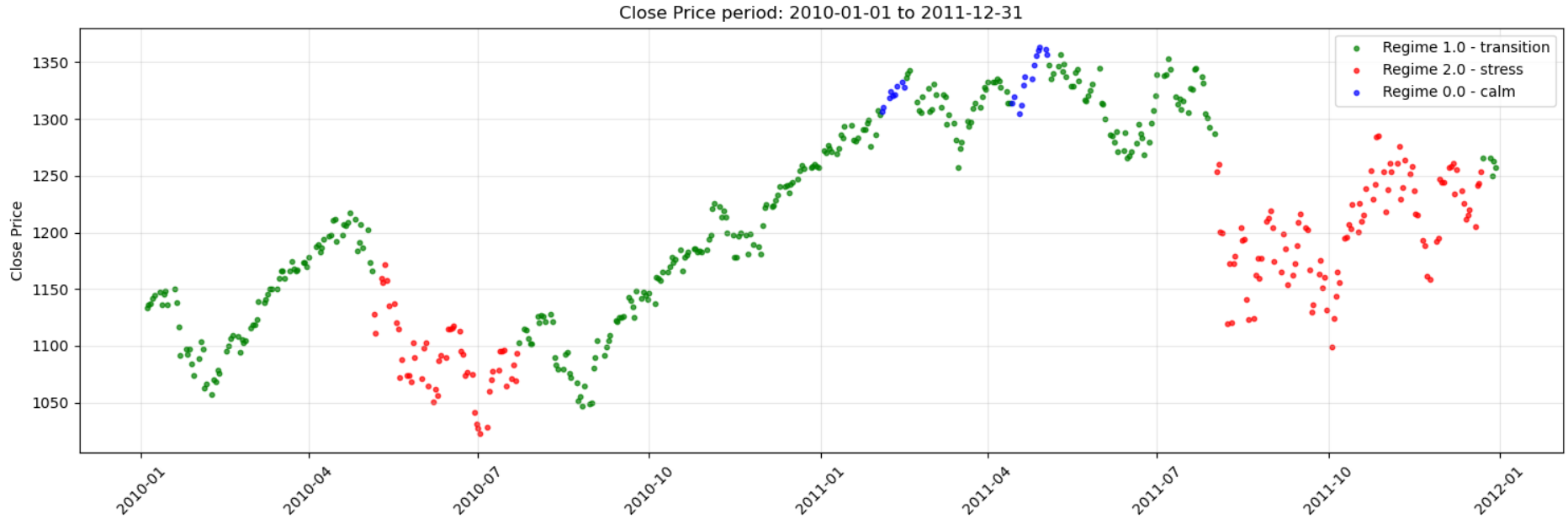
In [28]:
```
# Define your date ranges
date_ranges = [
    ('2000-01-01', '2001-12-31'),
    ('2002-01-01', '2003-12-31'),
    ('2004-01-01', '2005-12-31'),
    ('2006-01-01', '2007-12-31'),
    ('2008-01-01', '2009-12-31'),
    ('2010-01-01', '2011-12-31'),
```

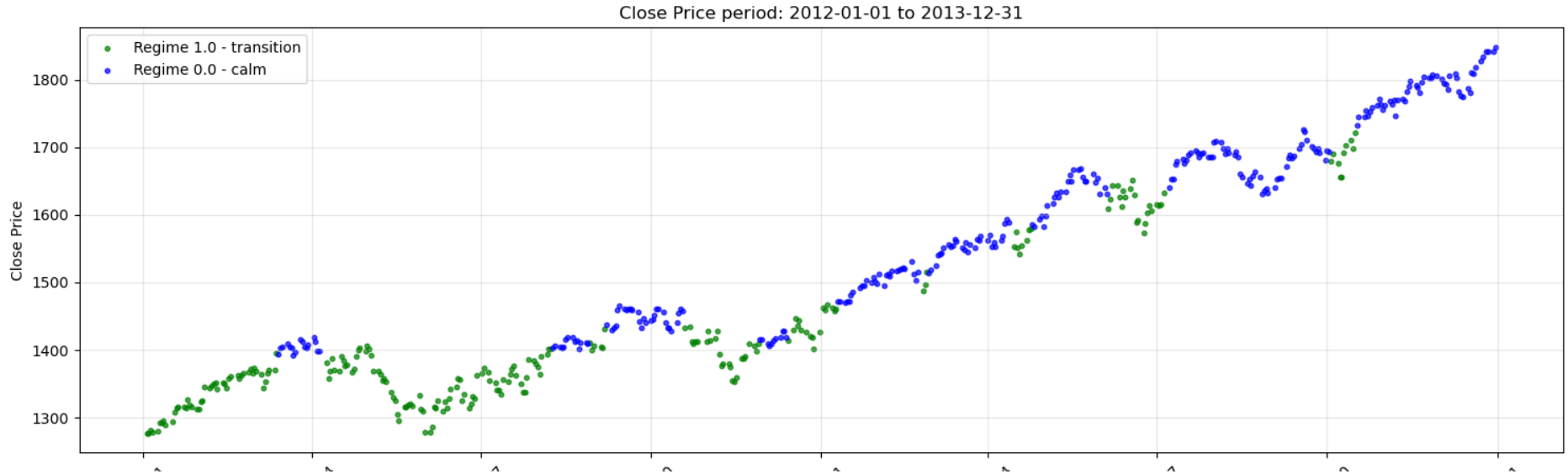
```python
    ('2012-01-01', '2013-12-31'),
    ('2014-01-01', '2015-12-31'),
    ('2016-01-01', '2017-12-31'),
    ('2018-01-01', '2019-12-31'),
    ('2020-01-01', '2021-12-31'),
    ('2022-01-01', '2023-12-31'),
    ('2024-01-01', '2025-12-31')
]

# Create subplots
n_plots = len(date_ranges)
fig, axes = plt.subplots(n_plots, 1, figsize=(15, 5*n_plots))

colors = {0: 'blue', 1: 'green', 2: 'red'}

for idx, (start_date, end_date) in enumerate(date_ranges):
    # Filter data for this date range
    mask = (combined_df.index >= start_date) & (combined_df.index <= end_date)
    df_range = combined_df[mask]

    if len(df_range) > 0:
        ax = axes[idx]

        # Plot each regime
        for regime in df_range['hmm_regime'].unique():
            regime_mask = df_range['hmm_regime'] == regime
            if regime == 0:
                regime_name = "calm"
            elif regime == 1:
                regime_name = "transition"
            else:
                regime_name = "stress"

            ax.scatter(df_range.index[regime_mask],
                       df_range['Close'][regime_mask],
                       c=colors.get(regime, 'gray'),
                       label=f'Regime {regime} - {regime_name}',
                       s=10, alpha=0.7)
        # Add legend
        ax.legend()

        ax.set_title(f'Close Price period: {start_date} to {end_date}', fontsize=12)
        ax.set_ylabel('Close Price', fontsize=10)
        ax.grid(True, alpha=0.3)
        ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

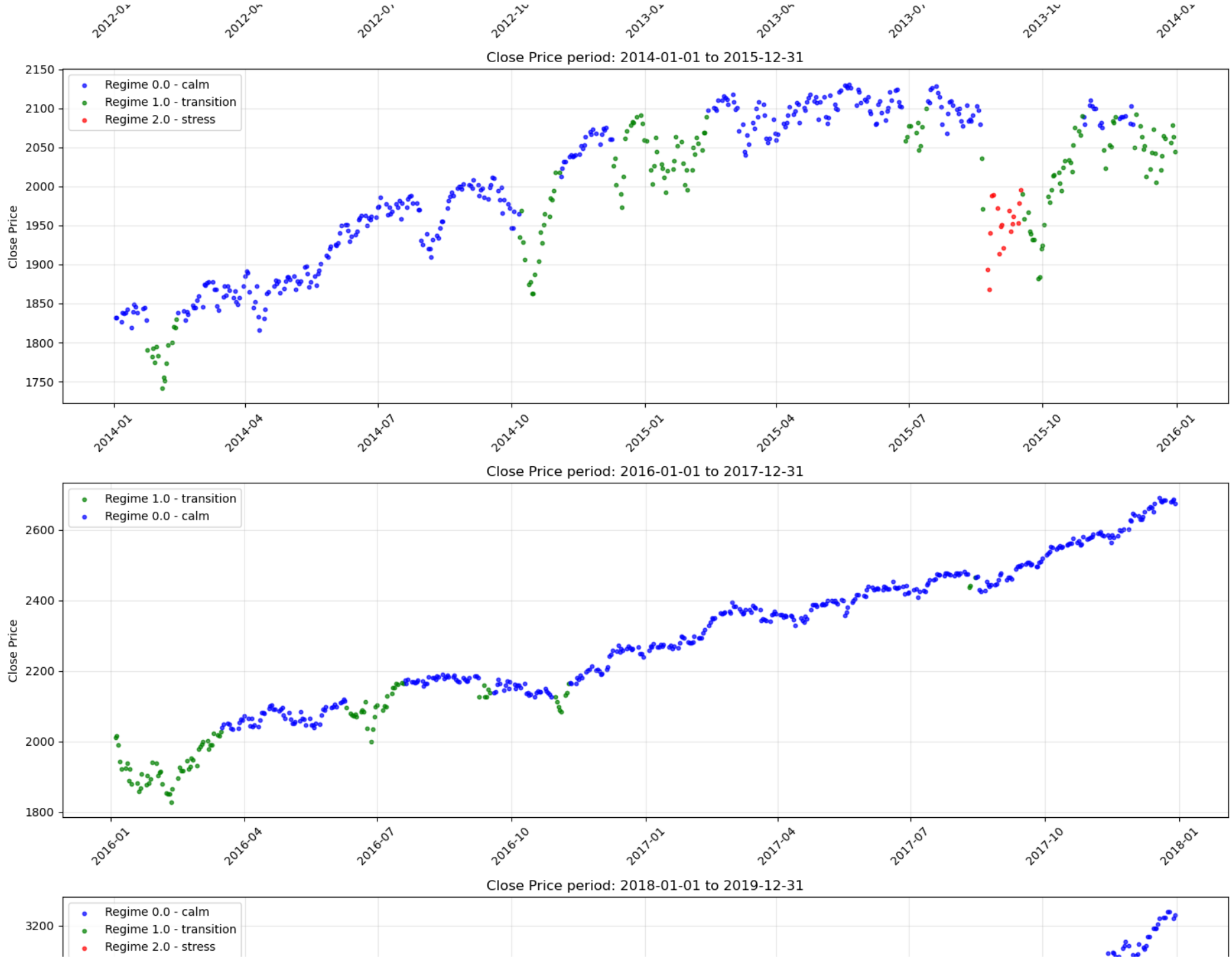

Close Price period: 2000-01-01 to 2001-12-31
Regime 2.0 - stress
Regime 1.0 - transition
Close Price
1500
1400
1300
1200
1100
1000
2000-05
2000-07
2000-09
2000-11
2001-01
2001-03
2001-05
2001-07
2001-09
2001-11
2002-01
Close Price period: 2002-01-01 to 2003-12-31
Regime 1.0 - transition
Regime 2.0 - stress
Close Price
1150
1100
1050
1000
950
900
850
800
2002-01
2002-04
2002-07
2002-10
2003-01
2003-04
2003-07
2003-10
2004-01
Close Price period: 2004-01-01 to 2005-12-31
Regime 1.0 - transition
Regime 0.0 - calm
1250

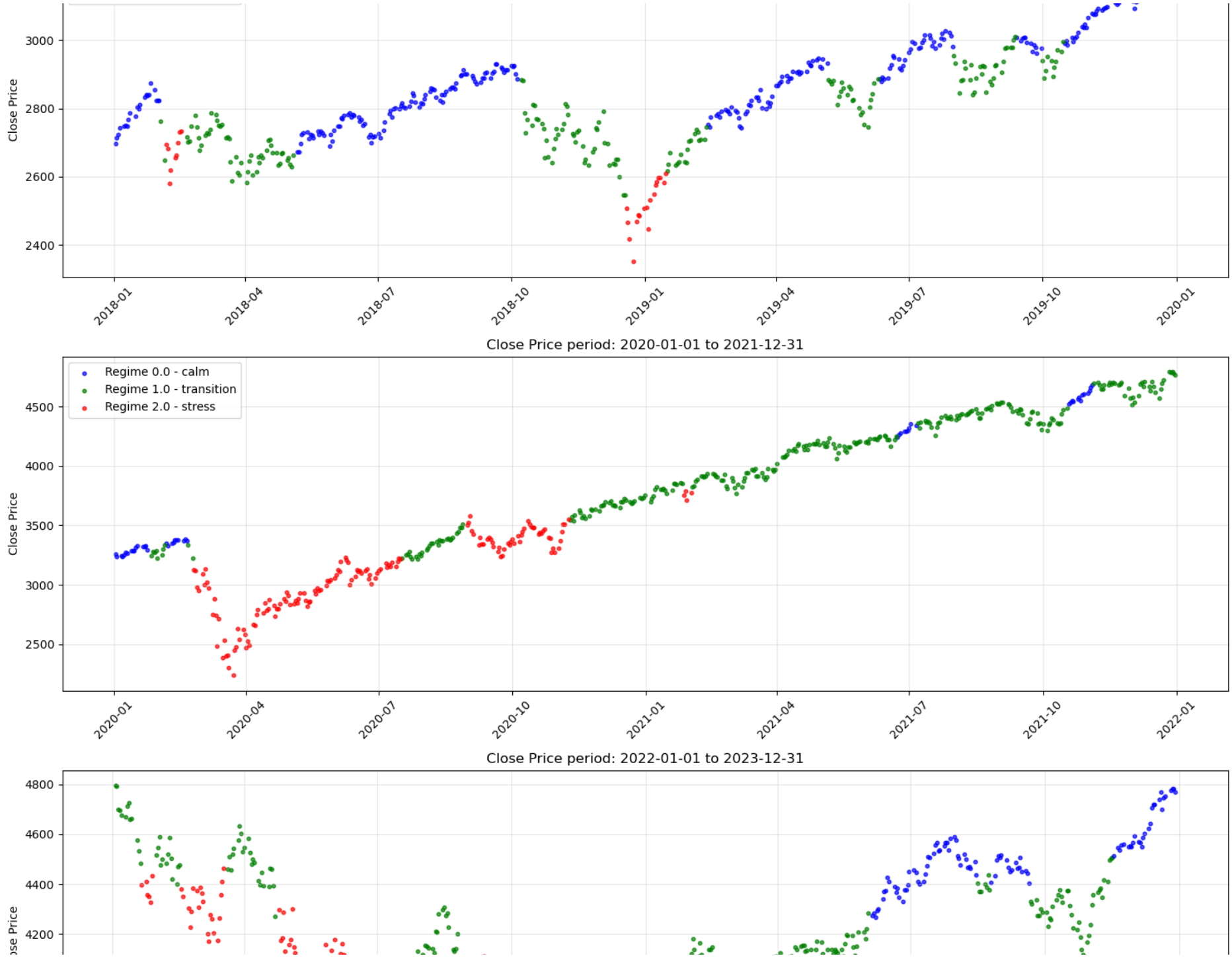

Close Price
1200
1150
1100
2004-01
2004-04
2004-07
2004-10
2005-01
2005-04
2005-07
2005-10
2006-01
Close Price period: 2006-01-01 to 2007-12-31
Regime 0.0 - calm
Regime 1.0 - transition
Regime 2.0 - stress
Close Price
1550
1500
1450
1400
1350
1300
1250
2006-01
2006-04
2006-07
2006-10
2007-01
2007-04
2007-07
2007-10
2008-01
Close Price period: 2008-01-01 to 2009-12-31
Regime 1.0 - transition
Regime 2.0 - stress
Close Price
1400
1300
1200
1100
1000

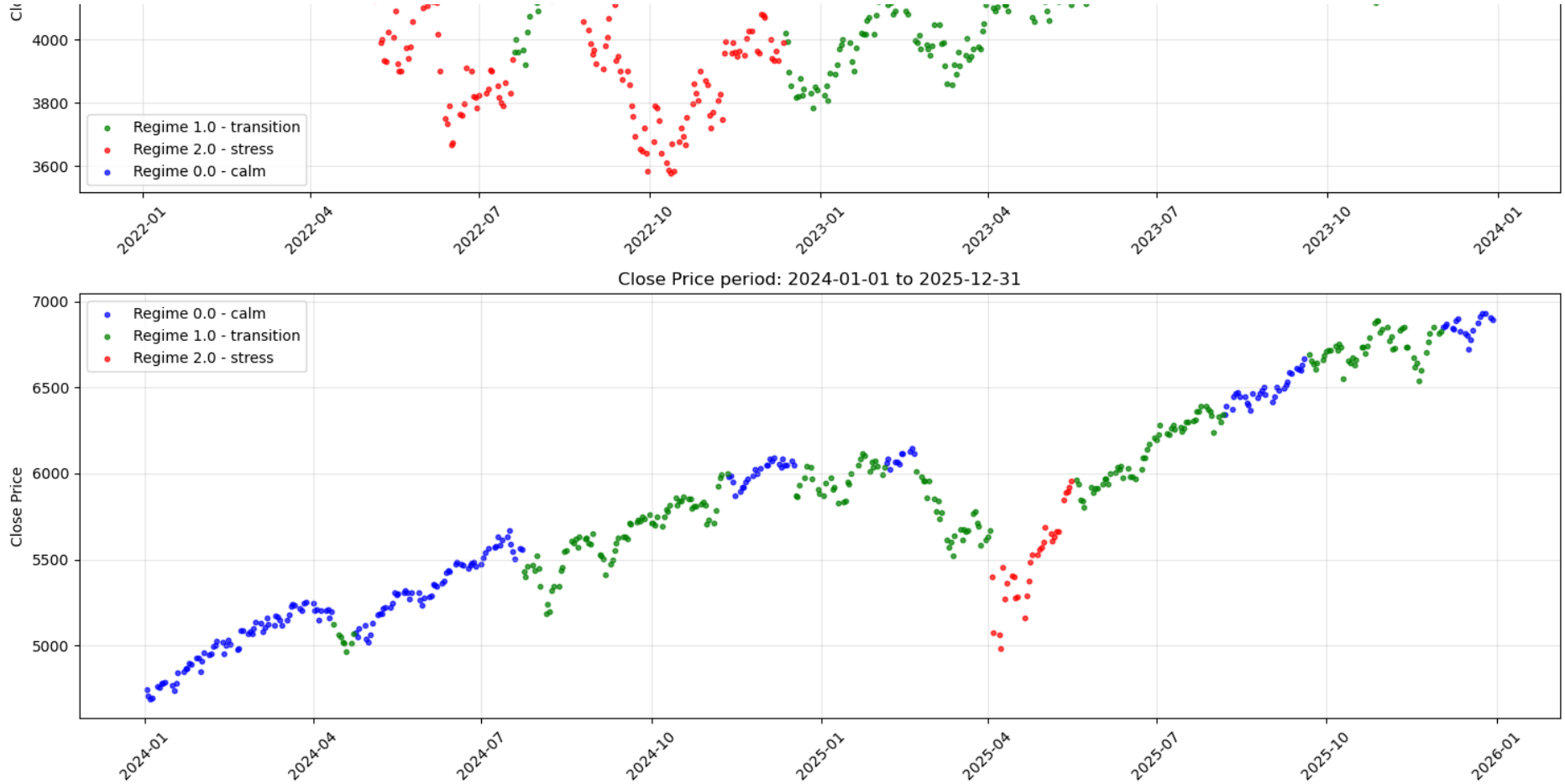

900
800
700
2008-01
2008-04
2008-07
2008-10
2009-01
2009-04
2009-07
2009-10
2010-01


Close Price period: 2010-01-01 to 2011-12-31
Regime 1.0 - transition
Regime 2.0 - stress
Regime 0.0 - calm
Close Price
1350
1300
1250
1200
1150
1100
1050
2010-01
2010-04
2010-07
2010-10
2011-01
2011-04
2011-07
2011-10
2012-01


Close Price period: 2012-01-01 to 2013-12-31
Regime 1.0 - transition
Regime 0.0 - calm
Close Price
1800
1700
1600
1500
1400
1300


Close Price period: 2014-01-01 to 2015-12-31
Regime 0.0 - calm
Regime 1.0 - transition
Regime 2.0 - stress
Close Price
2150
2100
2050
2000
1950
1900
1850
1800
1750
2014-01
2014-04
2014-07
2014-10
2015-01
2015-04
2015-07
2015-10
2016-01
Close Price period: 2016-01-01 to 2017-12-31
Regime 1.0 - transition
Regime 0.0 - calm
Close Price
2600
2400
2200
2000
1800
2016-01
2016-04
2016-07
2016-10
2017-01
2017-04
2017-07
2017-10
2018-01
Close Price period: 2018-01-01 to 2019-12-31
Regime 0.0 - calm
Regime 1.0 - transition
Regime 2.0 - stress
3200


Close Price
3000
2800
2600
2400
2018-01
2018-04
2018-07
2018-10
2019-01
2019-04
2019-07
2019-10
2020-01
Close Price period: 2020-01-01 to 2021-12-31
Regime 0.0 - calm
Regime 1.0 - transition
Regime 2.0 - stress
Close Price
4500
4000
3500
3000
2500
2020-01
2020-04
2020-07
2020-10
2021-01
2021-04
2021-07
2021-10
2022-01
Close Price period: 2022-01-01 to 2023-12-31
4800
4600
4400
4200



In [ ]:
```
#==========================================================================================
```

In [ ]:

Vetting Features - Factors

In [30]:
```
# Copy
vet_features_df = hmm_features_df.copy(deep=True)
```

In [31]:
```
# Helper Functions
# IC - Information Coefficient - function (Spearman)
    #-------------------------------------
    # IC(feature, fwd_return | regime=calm)
```

```
    # IC(feature, fwd_return | regime=stress)
    #--------------------------------------
def ic(x, y):
    return pd.Series(x).corr(pd.Series(y), method='spearman')

# Lead-lag IC
def lead_lag_ic(feature, returns, lags=range(-5,6)):
    out={}
    for lag in lags:
        shifted_ret = returns.shift(-lag)
        out[lag] = ic(feature, shifted_ret)
    return out
```

In [32]:
```
vet_features_df.shape
```

Out[32]: (6468, 68)

In [33]:
```
check_nan(vet_features_df)
# Clean
vet_features_df = vet_features_df.dropna()
```

```
Does the DataFrame contain any NaN values? False
```

In [34]:
```
# List of factors to exlude from vettting
col_to_exclude = [
    'Volume',
 'return',
 'fwd_return',
 'log_ret',
 'realized_vol',
 'VIX_level',
 'VIX_change',
 'VIX_indx_close',
 'vol_z',
 'hmm_regime',
 'regime_name'
]
# Vetting factors
col_to_keep = [col for col in vet_features_df.columns if col not in col_to_exclude]
```

In [35]:
```
# Feature Vetting Function
def vet_feature(df, feature_col, n_boot=500, window=126):

#=====================================================
    def rolling_spearman_ic(f, r, window):

        ic_s = []
        for i in range(len(f)):
            if i < window:
```

```
                ic_s.append(np.nan)
                continue

            f_win = f.iloc[i-window:i]
            r_win = r.iloc[i-window:i]

            #-------------------------
            if f_win.nunique() < 2 or r_win.nunique() < 2:
                ic = np.nan
            else:
                ic = f_win.corr(r_win, method="spearman")

            ic_s.append(ic)

            # ic = f_win.corr(r_win, method="spearman")
            # ic_s.append(ic_s)

        return pd.Series(ic_s, index=f.index)
#=====================================================

    f = df[feature_col]
    r = df["fwd_return"]

    # align
    paired = pd.concat([f,r],axis=1).dropna()
    paired.columns=["f","r"]

    f = paired["f"]
    r = paired["r"]

    regime = df.loc[paired.index, "hmm_regime"]
    vol_z  = df.loc[paired.index, "vol_z"]

    # --- Overall IC
    ic_all = ic(f, r)

#-------------------------------------------------------------------------------
    # ---- IC (Information Coefficient) significance test
    n = len(paired)

    if n>3 and abs(ic_all)<1:
        t_stat = ic_all*np.sqrt((n-2)/(1-ic_all**2))
        p_value = 2*(1-stats.t.cdf(abs(t_stat),df=n-2))
    else:
        t_stat,p_value = np.nan,np.nan
#-------------------------------------------------------------------------------

# ---- Rolling IC.
    # Measures time stability.
```

```
    ric = rolling_spearman_ic(f, r, window)
    ric = ric.dropna()

    if len(ric)==0:
        stability_ric = np.nan
    else:
        mean_ric = ric.mean()
        std_ric = ric.std()
        # Stability metric. Higher = more stable.
        stability_ric = abs(mean_ric)/(std_ric+1e-6)
#-------------------------------------------------------------------------------

# --_- Bootstrap IC.
    # Measures statistical reliability.

    ics = []
    for _ in range(n_boot):
        sample = paired.sample(len(paired),replace=True)
        icc = sample["f"].corr(sample["r"], method="spearman")
        if np.isnan(icc):
            continue
        ics.append(icc)

    bic = np.array(ics)

    mean_bic = bic.mean()
    ci_low, ci_high = np.percentile(bic, [5,95])
    interval_ci = ci_high - ci_low

#-------------------------------------------------------------------------------

# --- Regime ICs
    mask_calm = regime == 0        # calm
    mask_transition = regime == 1  # transition is ignored for now
    '''
    Transition regimes are often noise/mixed state.
    Option A — Ignore.Just remove it.
    Option B — We can use it as stability check:
    ic_transition = ic(f[mask_transition], r[mask_transition])
    If transition IC is wildly different from calm/stress --> instability flag.
    '''
    mask_stress = regime == 2      # stress

    # Minimum sample per regime rule
    min_n = 50
    if mask_calm.sum() >= min_n:
        ic_calm = ic(f[mask_calm], r[mask_calm])
    else:
```

```
        ic_calm = np.nan

    if mask_stress.sum() >= min_n:
        ic_stress = ic(f[mask_stress], r[mask_stress])
    else:
        ic_stress = np.nan

    if not np.isnan(ic_calm) and not np.isnan(ic_stress):
        regime_gap = abs(ic_calm - ic_stress)
    else:
        regime_gap = np.nan

    #print(f'mask_calm.sum(): {mask_calm.sum()}')
    #print(f'mask_stress.sum(): {mask_stress.sum()}')
    #print(f'==================================')

#-------------------------------------------------------------------------------

# --- Lead-lag IC
    ll = lead_lag_ic(f, r)
    peak_lag = max(ll, key=lambda k: abs(ll[k]))

#-------------------------------------------------------------------------------

# --- Volume conditioning (collider test)
    high_vol = vol_z > vol_z.median()
    low_vol = vol_z <= vol_z.median()
    ic_high_vol = ic(f[high_vol], r[high_vol])
    ic_low_vol = ic(f[low_vol], r[low_vol])

    vol_sensitivity = abs(ic_high_vol - ic_low_vol)

# --- Extreme regime dependence
    # A factor that flips by regime is not stable alpha — it’s conditional behavior.
    gap_IC = regime_gap > abs(ic_all)

    return {
        "ic_all": ic_all,                       # 2
        "ic_calm": ic_calm,
        "ic_stress": ic_stress,
        "regime_gap": regime_gap,
        "peak_lag": peak_lag,
        "vol_sensitivity": vol_sensitivity,
        "gap_gt_ic": gap_IC,
        "t-significance_test": t_stat,
        "abs.t-significance_test": abs(t_stat),
        "p_value": p_value,
        "stability_metric": stability_ric,      # 1
        "bootstrap_IC_mean": mean_bic,          # 3
```

```
        "ci_contains_zero":(ci_low<=0<=ci_high), # If TRUE → weak feature.
        "bootstrap_IC_interval": interval_ci
    }
```

In [36]:
```
# Run on All Features Vetting
feature_cols = col_to_keep.copy()
results = []

for col in feature_cols:
    res = vet_feature(vet_features_df, col)
    res["feature"] = col
    results.append(res)

results_df = pd.DataFrame(results)
```

In [37]:
```
# Factors Scoring
robust_score_df = results_df.copy(deep=True)
```

In [38]:
```
robust_score_df.shape
```

Out[38]:
```
(57, 15)
```

In [39]:
```
'''
Metric Combo:
|IC|
× rolling stability
× CI tightness
'''
# Robust Factor Score - Option 3
robust_score_df["robust_score_3"] = (
    robust_score_df["bootstrap_IC_mean"].abs()
    * robust_score_df["stability_metric"]
    * robust_score_df["bootstrap_IC_interval"]**(-1)
)
```

In [41]:
```
# option - Weights for features
robust_score_df['weight_3'] = robust_score_df['robust_score_3'].clip(lower=0)
```

In [42]:
```
#========= PRO Factor Score ==================#
```

In [43]:
```
# Robust Factor Score - Option 4
'''
A realistic composite score:
score = |IC|**a × Stability**b × Reliability**c × RegimeRobustness**d × LiquidityRobustness**e
'''
def factor_score(row, a=1, b=1.5, c=1, d=1, e=0.5):
    strength = abs(row["bootstrap_IC_mean"])**a
```

```
    stability = (
        row["stability_metric"] if row["stability_metric"]>0 else 0
    )**b

    reliability = (1/(row["bootstrap_IC_interval"]+1e-6)
    )**c

    regime = (1/(1+row["regime_gap"])
    )**d

    liquidity = (1/(1+row["vol_sensitivity"])
    )**e

    return (
        strength
        * stability
        * reliability
        * regime
        * liquidity
    )

robust_score_df["pro_score"] = robust_score_df.apply(factor_score, axis=1)
robust_score_df = robust_score_df.sort_values("pro_score", ascending=False)
```

In [44]:
```
# Fitering Sorting Metrics-Factors data set
robust_score_df_sorted = robust_score_df[['feature', 'pro_score', 'robust_score_3', 'ic_all', 'ic_calm', 'ic_stress', 'regime_gap', 'vol_s
                  'stability_metric', 'bootstrap_IC_mean', 'bootstrap_IC_interval','ci_contains_zero', 'gap_gt_ic']] \
    .sort_values("pro_score", ascending=False).reset_index().drop(columns='index')
```

In [45]:
```
robust_score_df_sorted.shape
```

Out[45]: (57, 13)

In [51]:
```
robust_score_df_sorted
```

Out[51]:

	feature	pro_score	robust_score_3	ic_all	ic_calm	ic_stress	regime_gap	vol_sensitivity	stability_metric	bootstrap_IC_mean	bootstrap_I
0	RSI_7_norm	2.098176	1.868698	-0.056772	-0.082330	-0.049543	0.032787	0.034777	1.391535	-0.057173	
1	RSI_14_norm	1.948817	1.711771	-0.048241	-0.079805	-0.041717	0.038087	0.041967	1.455437	-0.048700	
2	price_vs_EMA20	1.618108	1.431211	-0.044146	-0.066322	-0.044277	0.022045	0.032597	1.378795	-0.043954	
3	price_vs_EMA10	1.569930	1.496106	-0.050070	-0.064675	-0.050893	0.013782	0.021256	1.155797	-0.050422	
4	price_vs_SMA50	1.361215	1.132731	-0.032620	-0.058681	-0.036630	0.022052	0.036608	1.563794	-0.032166	
5	BB_position	1.305819	1.281385	-0.047682	-0.071430	-0.035603	0.035827	0.042795	1.161985	-0.046713	
6	high_close_ratio	1.265156	1.311002	0.059749	0.079496	0.085627	0.006132	0.007601	0.949948	0.060148	
7	price_vs_SMA20	1.166035	1.084484	-0.040647	-0.063137	-0.036647	0.026491	0.037935	1.264372	-0.040088	
8	ROC_10	1.060929	1.052120	-0.041829	-0.049404	-0.035603	0.013801	0.033933	1.080590	-0.041503	
9	close_position	0.928390	1.092066	-0.059944	-0.056987	-0.081749	0.024761	0.003029	0.761278	-0.060401	
10	body	0.590425	0.742384	-0.054400	-0.059885	-0.078864	0.018979	0.005948	0.660690	-0.053040	
11	term_spread	0.583652	0.709328	-0.034930	0.007386	-0.044907	0.052294	0.010982	0.757975	-0.035071	
12	ROC_20	0.565677	0.565441	-0.023820	-0.021263	-0.023014	0.001750	0.042218	1.046792	-0.023790	
13	OBV_base_dif	0.552298	0.728772	-0.048295	-0.032732	-0.101964	0.069232	0.006158	0.660684	-0.048425	
14	MACD_norm	0.530578	0.476133	-0.016815	-0.039437	-0.013151	0.026285	0.040982	1.361574	-0.015927	
15	daily_range	0.499530	0.543760	0.024341	0.076984	0.024921	0.052063	0.026507	0.958902	0.025097	
16	MACD_diff_norm	0.463549	0.527545	-0.027411	-0.047950	-0.017371	0.030579	0.036611	0.850098	-0.028853	
17	Low_return	0.404935	0.557358	-0.045509	-0.062914	-0.036409	0.026504	0.000587	0.556541	-0.046184	
18	news_sentiment	0.346537	0.433111	-0.026955	-0.030185	-0.030884	0.000699	0.002543	0.642736	-0.026513	
19	trend_strength_ema	0.316896	0.291566	-0.010348	-0.034468	-0.013393	0.021076	0.021064	1.257612	-0.010529	
20	High_return	0.205816	0.334056	-0.035479	-0.025556	-0.068982	0.043426	0.033479	0.427131	-0.035754	
21	low_close_ratio	0.202782	0.358325	0.043082	0.014951	0.068892	0.053941	0.003931	0.357160	0.043175	
22	Open_return	0.196036	0.308688	-0.034085	-0.028109	-0.023033	0.005076	0.010821	0.411835	-0.033772	
23	kin_f3_cpi_t10y3m	0.172964	0.226075	0.015745	0.037583	0.022323	0.015260	0.000697	0.603788	0.015458	
24	T10Y3M	0.169606	0.223651	-0.015419	-0.036986	-0.022503	0.014483	0.000328	0.592094	-0.015498	
25	upper_wick	0.167032	0.256509	0.027268	0.033900	0.025890	0.008010	0.001036	0.431311	0.027823	
26	EMA_cross	0.159481	0.210893	-0.014547	-0.037036	-0.002898	0.034138	0.028804	0.629216	-0.014663	
27	MACD_signal_norm	0.154542	0.149222	-0.005743	-0.027011	-0.004077	0.022934	0.024462	1.149844	-0.005575	

	feature	pro_score	robust_score_3	ic_all	ic_calm	ic_stress	regime_gap	vol_sensitivity	stability_metric	bootstrap_IC_mean	bootstrap_I
28	DGS10_dif	0.108606	0.195489	-0.025488	-0.025626	-0.050664	0.025038	0.017824	0.330091	-0.025249	
29	gap	0.106504	0.193419	-0.025472	-0.024635	-0.040042	0.015407	0.022405	0.319633	-0.025109	
30	MA_cross	0.072616	0.098062	-0.008091	-0.028354	0.002123	0.030477	0.016367	0.591841	-0.007373	
31	BB_width	0.058950	0.091292	0.010117	0.018623	0.019994	0.001371	0.051525	0.439669	0.009149	
32	range_pct_change	0.058178	0.118192	0.019180	0.035967	0.001657	0.034309	0.035948	0.268533	0.018193	
33	is_PAYEMS_release_day	0.041905	0.099072	-0.019729	0.011265	-0.035562	0.046827	0.034242	0.202785	-0.019496	
34	VIX_indx_close_return	0.036390	0.088043	0.018820	-0.006088	0.055570	0.061658	0.017406	0.195908	0.018392	
35	DTB3_dif	0.033388	0.069498	-0.013073	0.002905	0.001216	0.001689	0.004521	0.232634	-0.013130	
36	ADX_norm	0.031383	0.075756	0.018226	0.003324	-0.004333	0.007657	0.031311	0.179718	0.018674	
37	period_CPI	0.022162	0.056884	0.014778	0.016051	-0.006243	0.022294	0.025859	0.162736	0.014179	
38	DXY_indx_close_return	0.020662	0.053633	-0.012420	-0.022098	0.044359	0.066457	0.007463	0.170066	-0.012995	
39	gap_filled	0.019759	0.067580	0.030719	0.022897	0.042561	0.019664	0.003222	0.089175	0.031195	
40	Volume_return	0.015394	0.048470	-0.016380	0.038958	-0.048600	0.087557	0.019603	0.121643	-0.016844	
41	CPI_pct()	0.014678	0.042819	0.014914	0.026537	0.012522	0.014015	0.003539	0.121261	0.014359	
42	period_GDP	0.008141	0.024432	0.008624	-0.003957	0.012107	0.016063	0.013038	0.116134	0.008718	
43	T10Y2Y_yield_curve	0.007379	0.020918	-0.005763	0.024080	-0.019361	0.043441	0.020557	0.138266	-0.006256	
44	period_PAYMES_UNRATE	0.006620	0.023334	0.012465	0.008371	0.003206	0.005165	0.018946	0.082867	0.012347	
45	is_CPIAUCSL_release_day	0.006481	0.022048	0.009072	0.037675	-0.056400	0.094075	0.010420	0.104502	0.008823	
46	GDP_pct()	0.003331	0.014766	0.010721	0.001631	0.034129	0.032498	0.036631	0.056237	0.011926	
47	volume_trend	0.003012	0.009823	0.004067	-0.016021	-0.011568	0.004453	0.028761	0.097621	0.004410	
48	kin_f2_unr_f1	0.002347	0.016873	0.033988	0.035855	0.035722	0.000133	0.024796	0.019840	0.034659	
49	kin_f1_gdp_paym	0.001399	0.011531	-0.033885	-0.035697	-0.035548	0.000149	0.024711	0.015097	-0.034303	
50	PAYEMS_pct()	0.000880	0.003142	-0.001532	0.006530	0.005588	0.000942	0.001290	0.078756	-0.001582	
51	lower_wick	0.000770	0.002697	0.001595	0.026449	-0.001755	0.028204	0.023620	0.088253	0.001298	
52	is_GDP_release_day	0.000698	0.004155	0.006571	-0.009111	0.004690	0.013801	0.003809	0.029100	0.006432	
53	BB_squeeze	0.000634	0.003183	-0.003398	-0.012806	0.013660	0.026466	0.005577	0.042001	-0.003318	
54	volume_ratio	0.000603	0.004507	-0.008858	0.029789	-0.040305	0.070094	0.032325	0.021193	-0.009660	
55	T10Y2Y_dif	0.000490	0.001804	-0.002109	-0.022201	0.013558	0.035759	0.016717	0.080486	-0.001001	

	feature	pro_score	robust_score_3	ic_all	ic_calm	ic_stress	regime_gap	vol_sensitivity	stability_metric	bootstrap_IC_mean	bootstrap_I
56	UNRATE_pct()	0.000094	0.000666	-0.002163	-0.030538	0.019005	0.049543	0.013700	0.022201	-0.001218	

In [54]:
```
# Consider top tier factors
pro_score_df = robust_score_df_sorted.loc[0:len(robust_score_df_sorted)*0.5]
pro_score_df.to_csv("project_DATA/_pro_score_df.csv")
```

In [56]:
```
pro_score_df.set_index('feature', inplace=True)
pro_score_df.index.name = 'factors'
```

In [57]:
```
pro_score_df.shape
```

Out[57]: (29, 12)

In [58]:
```
pro_score_df
```

Out[58]:

factors	pro_score	robust_score_3	ic_all	ic_calm	ic_stress	regime_gap	vol_sensitivity	stability_metric	bootstrap_IC_mean	bootstrap_IC_interv
RSI_7_norm	2.098176	1.868698	-0.056772	-0.082330	-0.049543	0.032787	0.034777	1.391535	-0.057173	0.0425
RSI_14_norm	1.948817	1.711771	-0.048241	-0.079805	-0.041717	0.038087	0.041967	1.455437	-0.048700	0.0414
price_vs_EMA20	1.618108	1.431211	-0.044146	-0.066322	-0.044277	0.022045	0.032597	1.378795	-0.043954	0.0423
price_vs_EMA10	1.569930	1.496106	-0.050070	-0.064675	-0.050893	0.013782	0.021256	1.155797	-0.050422	0.0389
price_vs_SMA50	1.361215	1.132731	-0.032620	-0.058681	-0.036630	0.022052	0.036608	1.563794	-0.032166	0.0444
BB_position	1.305819	1.281385	-0.047682	-0.071430	-0.035603	0.035827	0.042795	1.161985	-0.046713	0.0423
high_close_ratio	1.265156	1.311002	0.059749	0.079496	0.085627	0.006132	0.007601	0.949948	0.060148	0.0435
price_vs_SMA20	1.166035	1.084484	-0.040647	-0.063137	-0.036647	0.026491	0.037935	1.264372	-0.040088	0.0467
ROC_10	1.060929	1.052120	-0.041829	-0.049404	-0.035603	0.013801	0.033933	1.080590	-0.041503	0.0426
close_position	0.928390	1.092066	-0.059944	-0.056987	-0.081749	0.024761	0.003029	0.761278	-0.060401	0.0421
body	0.590425	0.742384	-0.054400	-0.059885	-0.078864	0.018979	0.005948	0.660690	-0.053040	0.0472
term_spread	0.583652	0.709328	-0.034930	0.007386	-0.044907	0.052294	0.010982	0.757975	-0.035071	0.0374
ROC_20	0.565677	0.565441	-0.023820	-0.021263	-0.023014	0.001750	0.042218	1.046792	-0.023790	0.0440
OBV_base_dif	0.552298	0.728772	-0.048295	-0.032732	-0.101964	0.069232	0.006158	0.660684	-0.048425	0.0439
MACD_norm	0.530578	0.476133	-0.016815	-0.039437	-0.013151	0.026285	0.040982	1.361574	-0.015927	0.0455
daily_range	0.499530	0.543760	0.024341	0.076984	0.024921	0.052063	0.026507	0.958902	0.025097	0.0442
MACD_diff_norm	0.463549	0.527545	-0.027411	-0.047950	-0.017371	0.030579	0.036611	0.850098	-0.028853	0.0464
Low_return	0.404935	0.557358	-0.045509	-0.062914	-0.036409	0.026504	0.000587	0.556541	-0.046184	0.0461
news_sentiment	0.346537	0.433111	-0.026955	-0.030185	-0.030884	0.000699	0.002543	0.642736	-0.026513	0.0393
trend_strength_ema	0.316896	0.291566	-0.010348	-0.034468	-0.013393	0.021076	0.021064	1.257612	-0.010529	0.0454
High_return	0.205816	0.334056	-0.035479	-0.025556	-0.068982	0.043426	0.033479	0.427131	-0.035754	0.0457
low_close_ratio	0.202782	0.358325	0.043082	0.014951	0.068892	0.053941	0.003931	0.357160	0.043175	0.0430
Open_return	0.196036	0.308688	-0.034085	-0.028109	-0.023033	0.005076	0.010821	0.411835	-0.033772	0.0450
kin_f3_cpi_t10y3m	0.172964	0.226075	0.015745	0.037583	0.022323	0.015260	0.000697	0.603788	0.015458	0.0412
T10Y3M	0.169606	0.223651	-0.015419	-0.036986	-0.022503	0.014483	0.000328	0.592094	-0.015498	0.0410
upper_wick	0.167032	0.256509	0.027268	0.033900	0.025890	0.008010	0.001036	0.431311	0.027823	0.0467
EMA_cross	0.159481	0.210893	-0.014547	-0.037036	-0.002898	0.034138	0.028804	0.629216	-0.014663	0.0437

factors	pro_score	robust_score_3	ic_all	ic_calm	ic_stress	regime_gap	vol_sensitivity	stability_metric	bootstrap_IC_mean	bootstrap_IC_interv
MACD_signal_norm	0.154542	0.149222	-0.005743	-0.027011	-0.004077	0.022934	0.024462	1.149844	-0.005575	0.0429
DGS10_dif	0.108606	0.195489	-0.025488	-0.025626	-0.050664	0.025038	0.017824	0.330091	-0.025249	0.0426

How to treat ci_contains_zero and gap_gt_ic.

```
🔹 ci_contains_zero
Means:
Bootstrap CI includes 0 → IC may be noise
This is a statistical reliability flag.
It does NOT automatically mean useless.
It means uncertain.

🔹 gap_gt_ic
regime_gap > |ic_all|
Meaning:
Regime dependence stronger than overall signal
This is a stability / causality flag.
This one is actually more dangerous than CI.
A factor that flips by regime is not stable alpha — it's conditional behavior.
```

Decision Table

```
| ci_contains_zero | gap_gt_ic | Action |
|------------------|----------|--------|
| False            | False    | KEEP (strong & stable) |
| True             | False    | PENALIZE (weak evidence but stable) |
| False            | True     | PENALIZE HEAVILY or regime-split |
| True             | True     |  DROP |


(False, True) → Regime-dependent
Use as regime-conditional factor:

if regime == calm: use factor
    else: ignore
```

In [60]:
```
pro_score_df_copy = pro_score_df.copy(deep=True)
```

In [61]:
```
#================ Filter and Penalize weak parameters ==========================
def filter_and_penalties(df):
    # First filter: remove rows where both conditions are True or df['gap_gt_ic'] = True
    df = df[~((df['ci_contains_zero']) & (df['gap_gt_ic']))].copy(deep=True)
    df = df[~((df['gap_gt_ic']))].copy(deep=True)

     # Second filter: keep only rows with same sign in ic columns
    df_filtered = df[np.sign(df['ic_calm']) == np.sign(df['ic_stress'])].copy(deep=True)

    # Define penalty conditions
    conditions = [
        df_filtered['ci_contains_zero'] & ~df_filtered['gap_gt_ic'], # Only ci_contains_zero True
        ~df_filtered['ci_contains_zero'] & df_filtered['gap_gt_ic'], # Only gap_gt_ic True
    ]

    # Corresponding penalty values
    penalties = [0.6, 0.4]

    # Apply the appropriate penalty to each row
    df_filtered['pro_score_penal'] = np.select(conditions, penalties, default=1.0) * df_filtered['pro_score']

    # Check if regime_gap column exists
    if 'regime_gap' in df_filtered.columns:
        # Apply penalty row-by-row where regime_gap is NaN
        df_filtered.loc[df_filtered['regime_gap'].isna(), 'pro_score_penal'] *= 0.7
    else:
        print("Warning: 'regime_gap' column not found")

    return df_filtered
#==============================================================================================
```

In [62]:
```
# Using
pro_score_df_copy_penal = filter_and_penalties(pro_score_df_copy)
pro_score_df_copy_penal.sort_values('pro_score_penal', ascending=False, inplace=True)
```

In [63]:
```
# Optioanal
pro_score_df_copy_penal['score_delta'] = pro_score_df_copy_penal['pro_score'] - pro_score_df_copy_penal['pro_score_penal']
```

In [66]:
```
# Save for future use in part 1.3.
pro_score_df_copy_penal.to_csv("project_DATA/_pro_score_df_metrics_factors.csv")
vet_features_df.to_csv("project_DATA/_vet_features_df.csv")
pro_score_df_copy_penal.shape, vet_features_df.shape
```

Out[66]:
```
((19, 14), (6468, 68))
```

In [68]:
```
pro_score_df_copy_penal.T.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 14 entries, pro_score to score_delta
Data columns (total 19 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   RSI_7_norm         14 non-null     object
 1   RSI_14_norm        14 non-null     object
 2   price_vs_EMA20     14 non-null     object
 3   price_vs_EMA10     14 non-null     object
 4   price_vs_SMA50     14 non-null     object
 5   BB_position        14 non-null     object
 6   high_close_ratio   14 non-null     object
 7   price_vs_SMA20     14 non-null     object
 8   ROC_10             14 non-null     object
 9   close_position     14 non-null     object
 10  body               14 non-null     object
 11  ROC_20             14 non-null     object
 12  Low_return         14 non-null     object
 13  news_sentiment     14 non-null     object
 14  Open_return        14 non-null     object
 15  upper_wick         14 non-null     object
 16  DGS10_dif          14 non-null     object
 17  kin_f3_cpi_t10y3m  14 non-null     object
 18  T10Y3M             14 non-null     object
dtypes: object(19)
memory usage: 2.7+ KB
```

In [69]:
```
vet_features_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6468 entries, 2000-04-12 to 2025-12-30
Data columns (total 68 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Volume                   6468 non-null   int64
 1   VIX_indx_close           6468 non-null   float64
 2   term_spread              6468 non-null   float64
 3   T10Y3M                   6468 non-null   float64
 4   T10Y2Y_yield_curve       6468 non-null   int64
 5   DGS10_dif                6468 non-null   float64
 6   T10Y2Y_dif               6468 non-null   float64
 7   DTB3_dif                 6468 non-null   float64
 8   news_sentiment           6468 non-null   float64
 9   is_GDP_release_day       6468 non-null   int64
 10  is_CPIAUCSL_release_day  6468 non-null   int64
 11  is_PAYEMS_release_day    6468 non-null   int64
 12  OBV_base_dif             6468 non-null   float64
 13  period_CPI               6468 non-null   int64
 14  period_PAYMES_UNRATE     6468 non-null   int64
 15  period_GDP               6468 non-null   int64
 16  CPI_pct()                6468 non-null   float64
 17  GDP_pct()                6468 non-null   float64
 18  PAYEMS_pct()             6468 non-null   float64
 19  UNRATE_pct()             6468 non-null   float64
 20  kin_f1_gdp_paym          6468 non-null   float64
 21  kin_f2_unr_f1            6468 non-null   float64
 22  kin_f3_cpi_t10y3m        6468 non-null   float64
 23  DXY_indx_close_return    6468 non-null   float64
 24  Low_return               6468 non-null   float64
 25  Volume_return            6468 non-null   float64
 26  Open_return              6468 non-null   float64
 27  High_return              6468 non-null   float64
 28  VIX_indx_close_return    6468 non-null   float64
 29  gap                      6468 non-null   float64
 30  gap_filled               6468 non-null   int64
 31  daily_range              6468 non-null   float64
 32  range_pct_change         6468 non-null   float64
 33  close_position           6468 non-null   float64
 34  body                     6468 non-null   float64
 35  upper_wick               6468 non-null   float64
 36  lower_wick               6468 non-null   float64
 37  high_close_ratio         6468 non-null   float64
 38  low_close_ratio          6468 non-null   float64
 39  RSI_14_norm              6468 non-null   float64
 40  RSI_7_norm               6468 non-null   float64
 41  MACD_norm                6468 non-null   float64
 42  MACD_signal_norm         6468 non-null   float64
 43  MACD_diff_norm           6468 non-null   float64
```

```
 44  BB_width                 6468 non-null   float64
 45  BB_position              6468 non-null   float64
 46  BB_squeeze               6468 non-null   int64
 47  price_vs_EMA10           6468 non-null   float64
 48  price_vs_EMA20           6468 non-null   float64
 49  price_vs_SMA20           6468 non-null   float64
 50  price_vs_SMA50           6468 non-null   float64
 51  EMA_cross                6468 non-null   int64
 52  MA_cross                 6468 non-null   int64
 53  trend_strength_ema       6468 non-null   float64
 54  volume_ratio             6468 non-null   float64
 55  volume_trend             6468 non-null   float64
 56  ROC_10                   6468 non-null   float64
 57  ROC_20                   6468 non-null   float64
 58  ADX_norm                 6468 non-null   float64
 59  return                   6468 non-null   float64
 60  fwd_return               6468 non-null   float64
 61  log_ret                  6468 non-null   float64
 62  realized_vol             6468 non-null   float64
 63  VIX_level                6468 non-null   float64
 64  VIX_change               6468 non-null   float64
 65  vol_z                    6468 non-null   float64
 66  hmm_regime               6468 non-null   int64
 67  regime_name              6468 non-null   object
dtypes: float64(54), int64(13), object(1)
memory usage: 3.7+ MB
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: